

Milestone 4

Frontend Development, Deployment
& Application Interface

CliniScan — Chest X-ray Lung Abnormality Detection System

Author Lucky Soni	Batch B13 — CliniScan	Dataset VinBigData CXR	Frontend Next.js + TypeScript	Backend FastAPI + HuggingFace
-----------------------------	---------------------------------	----------------------------------	---	---

✓ Full-Stack Web Application Development	✓ Next.js + TypeScript Frontend
✓ FastAPI Backend on HuggingFace Spaces	✓ REST API Integration (4 Endpoints)
✓ Real-time Classification + Detection + Grad-CAM	✓ Batch Processing with CSV Export
✓ PDF Report Generation	✓ Production Deployment on Vercel

Live Application https://cliniscan-ashy.vercel.app	Frontend Repository github.com/luckysoni10/CliniScan_UI	Backend API luckysoni10-cliniscan-api.hf.space
--	---	---

TABLE OF CONTENTS

PART A — Project Overview & Milestone Objectives	3
PART B — System Architecture	3
B.1 Full-Stack Architecture Overview	3
B.2 Backend Architecture — FastAPI	4
B.3 Frontend Architecture — Next.js	4
PART C — Backend Development	5
C.1 Technology Stack	5
C.2 API Endpoints	5
C.3 Model Integration	5
C.4 Grad-CAM Implementation	6
PART D — Frontend Development	6
D.1 Technology Stack	6
D.2 Application Pages	7
D.3 Core Features	7
D.4 Component Architecture	8
PART E — Application Interface — Screenshots	8
PART F — Deployment	10
F.1 Backend Deployment — HuggingFace Spaces	10
F.2 Frontend Deployment — Vercel	10
F.3 Environment Configuration	11
PART G — API Integration & Testing	11
PART H — Deliverables & Project Completion	12
PART I — Conclusion	12

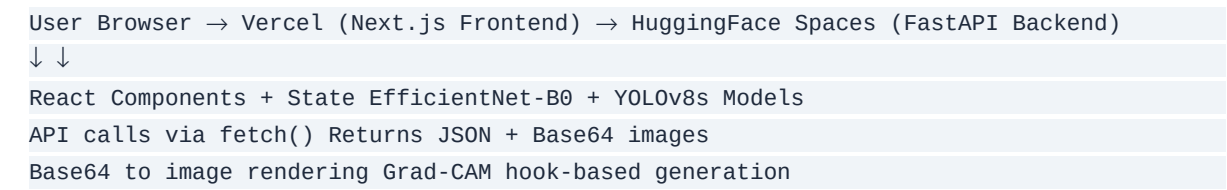
Milestone 4 Objective

Milestone 4 represents the final and most visible phase of the CliniScan project — transforming the trained AI models from Milestones 1–3 into a fully functional, publicly accessible web application. This milestone encompasses complete full-stack development, REST API design, model serving, and production deployment.

Objective	Implementation	Status
Backend API Server	FastAPI application with 4 prediction endpoints	✓ Complete
Model Serving	EfficientNet-B0 + YOLOv8s loaded via HuggingFace Hub	✓ Complete
Grad-CAM Heatmaps	Custom hook-based implementation for EfficientNet	✓ Complete
Frontend Application	Next.js 14 + TypeScript + Tailwind CSS	✓ Complete
Single Image Analysis	Classification + Detection + Grad-CAM workflows	✓ Complete
Batch Processing	Multi-file upload with CSV export	✓ Complete
PDF Report Generation	In-browser HTML print dialog with full results	✓ Complete
Backend Deployment	HuggingFace Spaces — Docker container	✓ Complete
Frontend Deployment	Vercel — Production URL publicly accessible	✓ Complete

B.1 Full-Stack Architecture Overview

CliniScan follows a decoupled client-server architecture where the frontend and backend are independently deployed on separate cloud platforms. This ensures scalability, maintainability, and allows each component to be updated independently.



B.2 Backend Architecture — FastAPI

The backend is a FastAPI application running inside a Docker container on HuggingFace Spaces. It loads both AI models at startup and exposes REST endpoints that accept multipart form data (image files) and return structured JSON responses.

Component	Technology	Purpose
Web Framework	FastAPI 0.115	REST API server with automatic OpenAPI docs
ASGI Server	Uvicorn	Production-grade async server on port 7860
Classification	PyTorch + EfficientNet-B0	Multi-label chest X-ray classification
Detection	Ultralytics YOLOv8s	Bounding box detection with annotated output
Interpretability	Custom GradCAM hooks	Gradient-weighted activation map generation
Model Storage	HuggingFace Hub	Weights downloaded at startup via hf_hub_download
CORS Middleware	FastAPI CORSMiddleware	Allow requests from Vercel frontend domain
Containerization	Docker	Reproducible deployment on HuggingFace Spaces

B.3 Frontend Architecture — Next.js

The frontend is a Next.js 14 application using the App Router, TypeScript for type safety, and Tailwind CSS for styling. It follows a single-page architecture with client-side state management using React hooks.

Component	Technology	Purpose
-----------	------------	---------

Framework	Next.js 14 (App Router)	React server/client components, routing
Language	TypeScript	Type-safe development, better IDE support
Styling	Tailwind CSS	Utility-first responsive styling
Animations	CSS Keyframes	Particle effects, transitions, scan lines
Icons	Lucide React	Consistent medical-themed iconography
State Management	React useState/useRef	Local component state for results, loading
API Communication	Native fetch()	FormData POST requests to FastAPI backend
Deployment	Vercel	Automatic Git-based CI/CD deployment

C.1 Technology Stack

Package	Version	Role
fastapi	0.115.0	Core web framework for REST API
uvicorn	0.30.0	ASGI server
torch + torchvision	2.1.0	Deep learning framework + EfficientNet
ultralytics	8.4.14	YOLOv8 model loading and inference
opencv-python-headless	4.9.0	Image processing and annotation
Pillow	10.3.0	Image loading and format conversion
huggingface_hub	0.23.0	Model weight download from HF Hub
python-multipart	0.0.9	Multipart form data parsing
numpy	1.26.4	Numerical operations for arrays

C.2 API Endpoints

Endpoint	Method	Input	Output
GET /health	GET	None	{'status':'ok','models':'loaded'}
POST /predict/classify	POST	image file, threshold	14 class probabilities + detected list
POST /predict/detect	POST	image file, confidence	Bounding boxes + base64 annotated image
POST /predict/gradcam	POST	image file	original_image + heatmap_image base64
POST /predict/batch	POST	multiple files, threshold	Array of per-file classification results

C.3 Model Integration

Both models are loaded at application startup using HuggingFace Hub download. Model weights are stored in the HuggingFace model repository 'luckysoni10/cliniscan-weights' and automatically downloaded when the container starts.

```
# Classification model
clf_model = efficientnet_b0(weights=None)
clf_model.classifier[1] = nn.Linear(clf_model.classifier[1].in_features, 14)
```

```
clf_path = hf_hub_download('luckysoni10/cliniscan-weights', 'm3_efficientnet_adamw.pth')
clf_model.load_state_dict(torch.load(clf_path, map_location='cpu'))

# Detection model
det_path = hf_hub_download('luckysoni10/cliniscan-weights', 'best.pt')
det_model = YOLO(det_path)
```

C.4 Grad-CAM Implementation

Gradient-weighted Class Activation Mapping is implemented using PyTorch forward and backward hooks registered on the last convolutional block of EfficientNet-B0. The heatmap is generated by weighting the feature map activations by their gradient importance and applying ReLU to retain only positive activations.

```
# Hook registration on EfficientNet last block
target_layer.register_forward_hook(→ store activations)
target_layer.register_full_backward_hook(→ store gradients)

# CAM generation
cam = sum(weight_i * activation_i for each channel)
cam = ReLU(cam) # Keep only positive values
cam = normalize(cam, 0 to 1)
heatmap = JET colormap overlay on original image (50/50 blend)
```

D.1 Technology Stack

Technology	Version	Purpose
Next.js	14.x	React framework with App Router and SSR/CSR support
TypeScript	5.x	Static typing for all components and API responses
Tailwind CSS	3.x	Utility-first CSS framework for responsive layout
Lucide React	0.383	Medical-themed SVG icon library
React	18.x	UI library — useState, useRef, useCallback hooks
CSS Keyframes	Native	Particle animations, scan lines, blob drift effects

D.2 Application Pages

Page	Route	Description
Landing Page	/	Hero section with animated particles, stats, feature cards, CTA buttons
Analysis	/app	Single X-ray upload, classification, detection, Grad-CAM results panel
Batch Process	/app	Multi-file upload, threshold config, results table, CSV export

D.3 Core Features

- **Landing Hero:** Full-viewport animated landing page with three drifting aurora blob particles, floating dot particles (CSS keyframes), gradient heading text, stats row (14+ pathologies, AUC 0.905, mAP50 0.176), and feature cards with hover effects.
- **X-ray Upload Zone:** Drag-and-drop + click-to-browse upload area supporting PNG, JPG, and DICOM formats. After upload, shows image preview with animated scan line overlay, file metadata, and clear button.
- **Analysis Configuration:** Classification threshold slider (0.1–0.9) with Sensitive/Balanced/Precise presets. Detection confidence slider with Low/Medium/High presets. Values persist across analysis runs.
- **Classification Results:** 14 animated probability bars with staggered entrance animation. Color-coded percentages (green/amber/red). Detected findings highlighted with warning icon. Chip row at bottom showing confirmed pathologies.
- **Detection Results:** Annotated X-ray image rendered from backend base64 response. Bounding box list with class names and confidence scores. Informational message when no boxes detected.

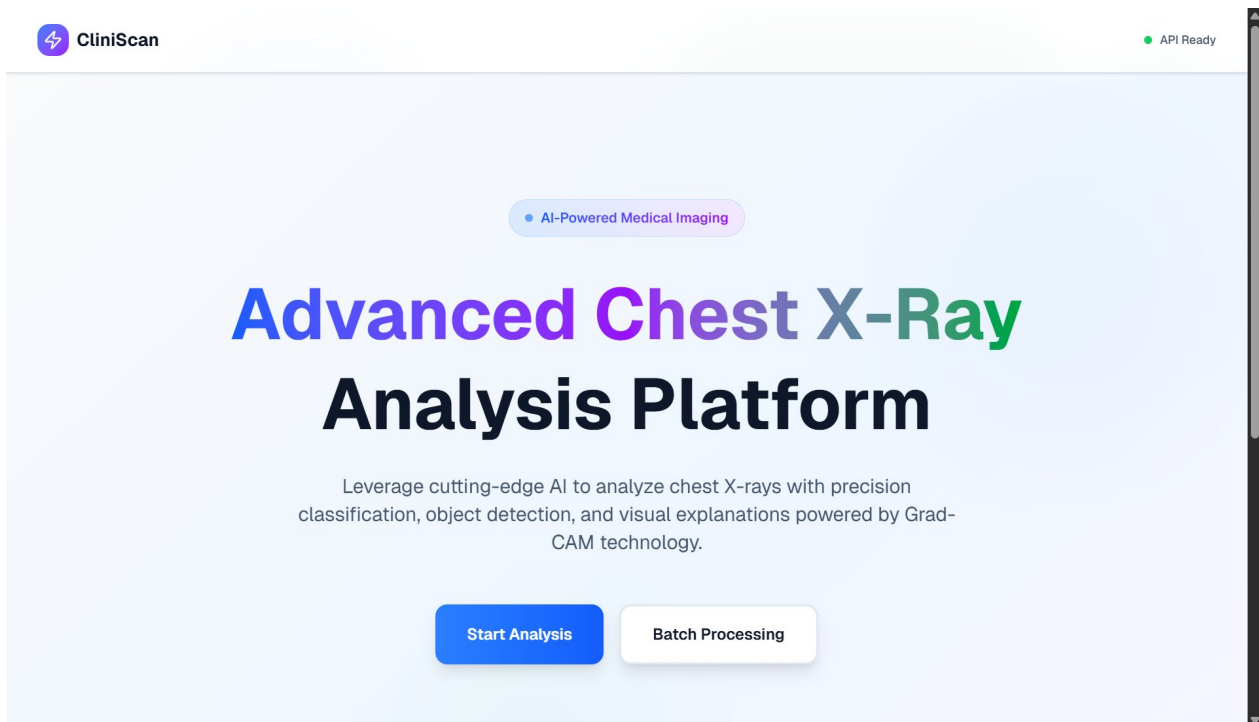
- **Grad-CAM Visualization:** Side-by-side original and heatmap display. Predicted class name and confidence shown above. Both images rendered from base64 API response.
- **Batch Processing:** Multi-file dropzone supporting up to 50 images. File grid showing thumbnails. Real-time processing with loading indicator. Results table with Normal/Abnormal badges and CSV download functionality.
- **PDF Report:** In-browser PDF generation triggered by `window.open + document.write + print()`. Includes classification results with probability bars, detection boxes, Grad-CAM predictions, and model performance metrics.
- **Toast Notifications:** Fixed bottom-right toast messages for API success and error states. Auto-dismiss after 3 seconds. Color-coded for success (green) and error (red).

D.4 Component Architecture

File / Component	Type	Responsibility
app/page.tsx	Client Page	Main application — landing + app pages, all state management
components/Particles.tsx	Client Component	CSS animated background particles (static values — SSR safe)
app/layout.tsx	Server Layout	Root layout with metadata and Google Fonts import
app/globals.css	Stylesheet	Tailwind base + custom CSS keyframes for animations
handleClassify()	Async Function	<code>fetch()</code> POST to <code>/predict/classify</code> , updates results state
handleDetect()	Async Function	<code>fetch()</code> POST to <code>/predict/detect</code> , renders annotated image
handleGradcam()	Async Function	<code>fetch()</code> POST to <code>/predict/gradcam</code> , renders heatmap
handleBatch()	Async Function	<code>fetch()</code> POST to <code>/predict/batch</code> with multiple files
generatePDF()	Function	Builds HTML report string, opens new window, triggers <code>print()</code>

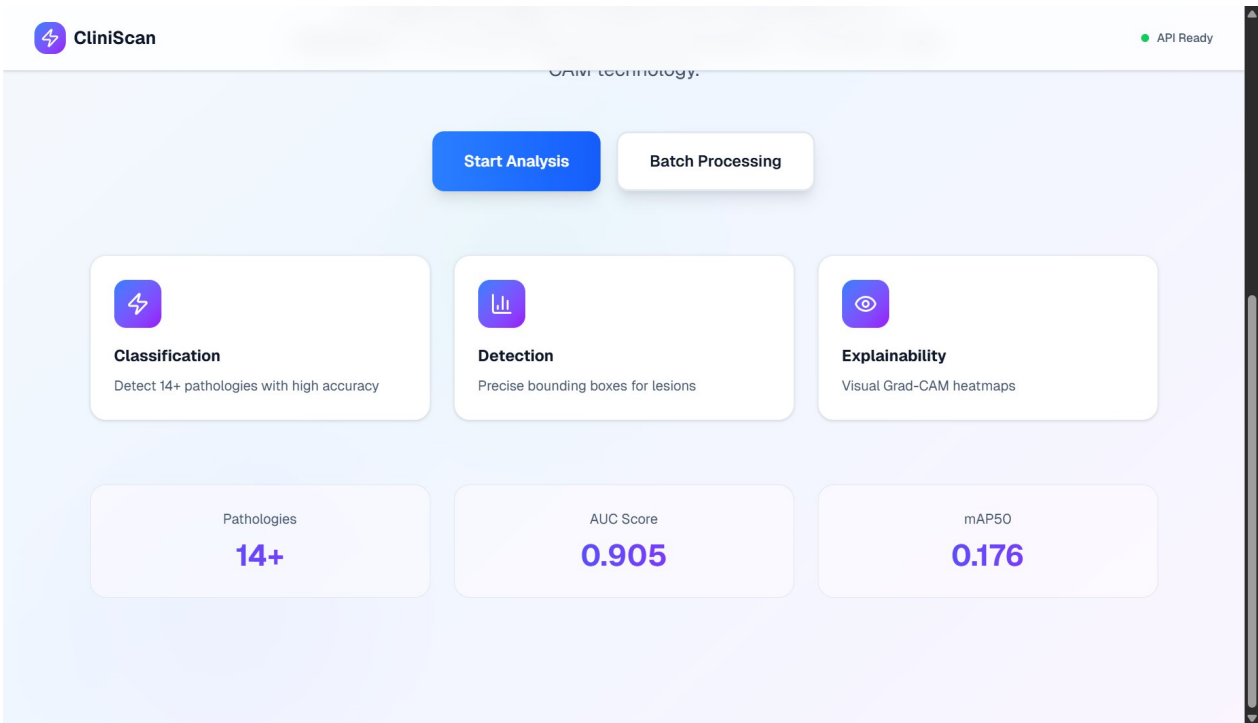
The following screenshots demonstrate the deployed CliniScan AI application running at <https://cliniscan-ashy.vercel.app>

Screenshot 1 — Landing Page Hero Section



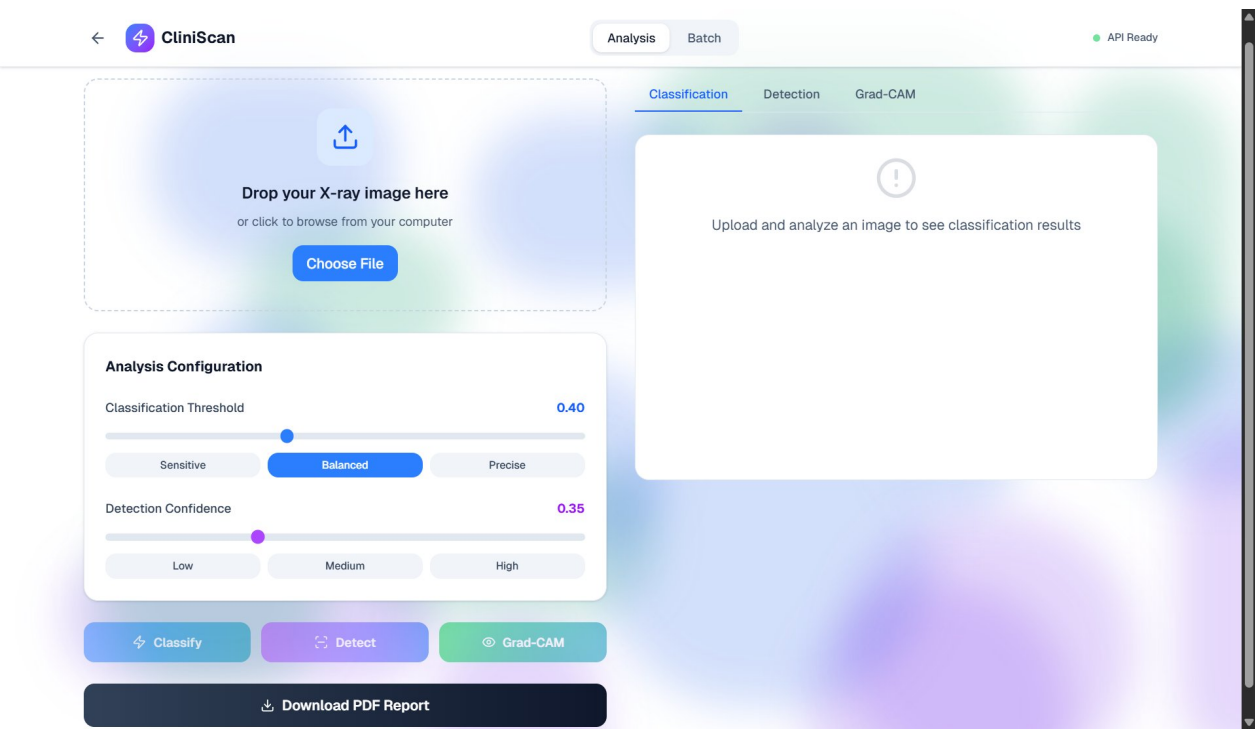
Landing page showing animated gradient heading, CTA buttons, and API status indicator.

Screenshot 2 — Feature Cards & Model Statistics



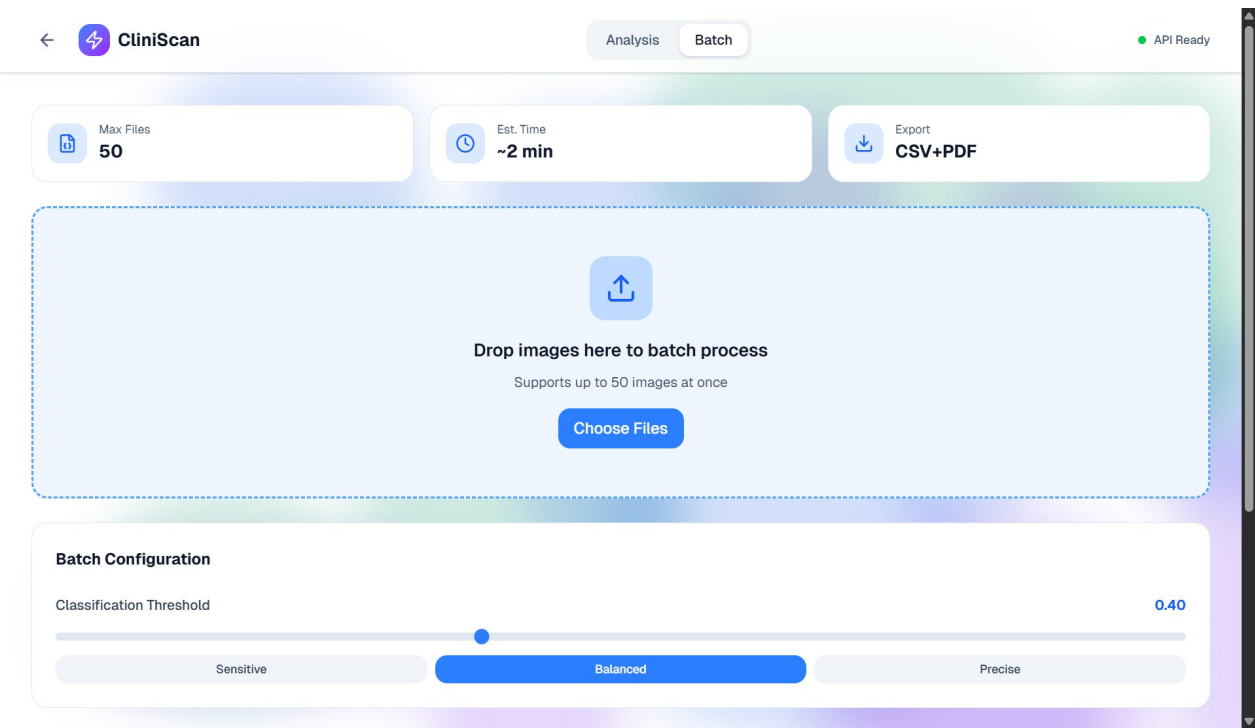
Feature cards (Classification, Detection, Explainability) and model performance stats (14+ pathologies, AUC 0.905, mAP50 0.176).

Screenshot 3 — Analysis Page (Single X-ray)



Analysis page showing upload zone, Analysis Configuration panel with sliders and presets, action buttons (Classify, Detect, Grad-CAM), PDF download button, and results panel with Classification/Detection/Grad-CAM tabs.

Screenshot 4 — Batch Processing Page



Batch processing page with info cards (Max Files 50, Est. Time ~2 min, Export CSV+PDF), multi-file dropzone, batch configuration threshold slider with presets, and Start Batch Analysis button.

F.1 Backend Deployment — HuggingFace Spaces

The FastAPI backend is deployed on HuggingFace Spaces using a Docker container. HuggingFace Spaces provides free CPU compute with persistent storage for model weights.

```
Space URL: https://huggingface.co/spaces/luckysoni10/cliniscan-api
API URL: https://luckysoni10-cliniscan-api.hf.space
SDK: Docker
Port: 7860 (HuggingFace default)
Model Repo: luckysoni10/cliniscan-weights
```

Dockerfile configuration:

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y libgl1-mesa-glx libsm6 ...
COPY requirements.txt . && pip install -r requirements.txt
COPY . .
EXPOSE 7860
CMD ['uvicorn', 'app:app', '--host', '0.0.0.0', '--port', '7860']
```

Configuration	Value
Platform	HuggingFace Spaces — Docker SDK
Compute	Free CPU tier — AMD
Model Weights	luckysoni10/cliniscan-weights (HF Model Repository)
Files Required	app.py, requirements.txt, Dockerfile, README.md
Auto-restart	Enabled — Space restarts on push to main branch
Health Check	GET /health → {'status':'ok','models':'loaded'}

F.2 Frontend Deployment — Vercel

The Next.js frontend is deployed on Vercel with automatic CI/CD from the GitHub repository. Every push to the main branch triggers an automatic rebuild and deployment.

Configuration	Value
Platform	Vercel — Hobby (Free)
Live URL	https://cliniscan-ashy.vercel.app
GitHub Repo	github.com/luckysoni10/CliniScan_UI
Framework	Next.js (auto-detected by Vercel)
Build Command	next build

Output Directory	.next (auto)
Node Version	18.x
Auto Deploy	Yes — on every push to main branch

F.3 Environment Configuration

The only environment variable required in the frontend is the API base URL, which points to the HuggingFace Spaces backend.

```
# In Next.js (set in Vercel dashboard or .env.local):  
NEXT_PUBLIC_API_URL=https://luckysoni10-cliniscan-api.hf.space  
  
# In app/page.tsx (hardcoded for simplicity):  
const API_BASE = 'https://luckysoni10-cliniscan-api.hf.space'
```

API Request/Response Flow

Endpoint	Request Format	Response Keys	Used For
/predict/classify	FormData: file + threshold	detected[], all_probs[], threshold	14-class probability bars
/predict/detect	FormData: file + confidence	boxes[], total_found, annotated_image	Bounding box display
/predict/gradcam	FormData: file	predicted_class, confidence, original_image, heatmap_image	Side-by-side heatmap
/predict/batch	FormData: files[] + threshold	results[{filename, detected[], finding_count}]	Batch results table

Frontend API Call Pattern

All API calls follow the same pattern — FormData POST with native `fetch()`, error handling with `try/catch`, and state updates on success.

```
const fd = new FormData()
fd.append('file', imageFile) // actual File object
const res = await fetch(`${API_BASE}/predict/classify?threshold=${threshold}`,
{ method: 'POST', body: fd }
)
if (!res.ok) throw new Error(`HTTP ${res.status}`)
const data = await res.json()
setResults(prev => ({ ...prev, classify: data })))
```

CORS Configuration

The FastAPI backend is configured with `CORSMiddleware` allowing all origins (`allow_origins=['*']`). This enables the Vercel-hosted frontend to make cross-origin requests to the HuggingFace-hosted backend without any CORS errors.

API Health Verification

The application displays a green 'API Ready' indicator in the top navigation bar confirming the backend is online. The frontend checks backend availability via the `GET /health` endpoint on page load.

Milestone 4 Deliverables Checklist

- ✓ Milestone 4 PDF Report — theory, architecture, screenshots, deployment
- ✓ Frontend Code — github.com/luckysoni10/CliniScan_UI
- ✓ Backend Code — `cliniscan-api/` folder with `app.py`, `Dockerfile`, `requirements.txt`
- ✓ Deployment Files — `Dockerfile`, `vercel.json`, `requirements.txt`
- ✓ Live Application URL — <https://cliniscan-ashy.vercel.app>
- ✓ Backend API URL — <https://luckysoni10-cliniscan-api.hf.space>
- ✓ Classification endpoint — `/predict/classify` (14-class multi-label)
- ✓ Detection endpoint — `/predict/detect` (YOLOv8s bounding boxes)
- ✓ Grad-CAM endpoint — `/predict/gradcam` (EfficientNet heatmaps)
- ✓ Batch endpoint — `/predict/batch` (multi-image processing)
- ✓ PDF report generation — in-browser print dialog
- ✓ CSV export — batch results downloadable
- ✓ Responsive design — works on desktop and mobile browsers
- ✓ Particle animation — CSS-only, SSR-safe implementation

Complete Project Milestone Summary

Milestone	Focus Area	Key Deliverable	Status
Milestone 1	Data & Preprocessing	5,000 images, YOLO annotations	✓ Done
Milestone 2	Baseline Model Training	ResNet50 + YOLOv8s baseline	✓ Done
Milestone 3	Model Optimization	EfficientNet AUC 0.905, mAP 0.176	✓ Done
Milestone 4	Frontend + Deployment	https://cliniscan-ashy.vercel.app	✓ Done

Conclusion

Milestone 4 successfully delivers a complete, production-ready web application for AI-powered chest X-ray analysis. The CliniScan system is now fully deployed and publicly accessible, transforming the trained deep learning models into a practical clinical decision support tool.

The frontend provides an intuitive, animated interface built with Next.js and TypeScript, while the backend FastAPI server efficiently serves both the EfficientNet-B0 classification model and the YOLOv8s detection model from HuggingFace Spaces. The complete pipeline from DICOM preprocessing to web-based inference demonstrates a full end-to-end medical AI development cycle.

Key technical achievements of this milestone include: real-time multi-label classification with probability visualization, bounding box detection with annotated image output, Grad-CAM heatmap generation for model interpretability, batch processing with CSV export, and PDF report generation — all integrated into a single, responsive web application deployed at production scale.

Final Model Performance Summary

Model	Task	Metric	Score
EfficientNet-B0	Classification	Test AUC	0.9046
EfficientNet-B0	Classification	Test F1 (macro)	0.2429
EfficientNet-B0	Classification	Test Loss	0.1585
YOLOv8s	Detection	mAP50	0.1761
YOLOv8s	Detection	mAP50-95	0.0790
YOLOv8s	Detection	Recall	0.196

Live Application

CliniScan AI — Live Application

<https://cliniscan-ashy.vercel.app>